

CONTEXTGRAPH: REPAIR MEMORY FOR MISSING REQUIREMENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Past repairs record requirements that a new issue may leave unstated. We study how retrieving these requirements before editing changes a complete coding agent’s repair. In paired comparisons on three Django development tasks, repairs pass official verification with or without memory, but only the history-informed repairs satisfy the historical requirement together with the current one. The added behavior preserves query parameters, removes redundant uniqueness, or honors an explicit destination timezone. Content interventions show that a behavioral requirement can suffice without its historical implementation. They also expose loss of triggering conditions during use: the same requirement produces different repairs when agents reconstruct different inputs. Further executions identify incorrect applicability and excessive preservation. These findings motivate memory that communicates the missing behavior together with the conditions under which it is required.

1 INTRODUCTION

The issue description tells a coding agent what is currently reported as broken. It need not enumerate every behavior a complete repair should preserve. A past repair can supply a requirement absent from the new issue, allowing memory to improve a repair that already passes the target’s tests. The target’s success score can then remain unchanged while the repair gains a required behavior. This motivates measuring memory’s contribution at the level of the requirement it supplies.

Consider an administrative redirect that must preserve a deployment prefix. Changing its destination from `request.path_info` to `request.path` fixes that reported fault. However, appending a slash to the resulting path still discards query parameters. A different repair of the same API records this omitted behavior and replaces path concatenation with `request.get_full_path(force_append_slash=True)`. The useful information in this experience is the requirement to retain the query, which must hold together with the deployment-prefix requirement.

We investigate three questions: what a historical repair adds to the current issue, what information suffices to communicate that addition, and how its conditions can be lost during use. The unit of analysis is a current and historical requirement exercised together on an actual repair.

2 USING HISTORY TO COMPLETE THE REPAIR REQUIREMENTS

Supply the requirement during generation. A complete SWE-agent receives the target issue and retrieved memory before browsing the repository, editing code, running tools, and submitting a patch. Historical records contain source-partition issues and their reference patches. Frozen BM25 rankings or cosine similarity with Qwen3-Embedding-8B select the sources (Zhang et al., 2025). We supply the stored rule, original issue and patch, or source-generated behavioral and implementation components. The complete-agent comparisons use DeepSeek V4 Pro0423 through OpenRouter, temperature 0, with the same tools and budget within each comparison.

Measure the requirements together. We immediately evaluate every submitted repair with the official SWE-bench verifier (Jimenez et al., 2024). To measure the behavior contributed by the

Table 1: Joint current/historical behavior in complete-agent comparisons. Each displayed repair passes official verification. The columns compare the actual repairs with and without memory; the rows retain their distinct task requirements.

Target	Historical addition	No memory	Memory
Django14404	Preserve query string	Fail	Pass
Django12708	Delete redundant uniqueness	Fail	Pass
Django11138	Honor destination timezone	Fail	Pass

source repair, a separate writer generates a short check from its patch and relevant repaired code. A normal execution changing from false on the buggy source to true on the repaired source demonstrates the historical behavior on the supplied inputs. We then execute this check unchanged on the patched target. An issue check measures the reported requirement, and a joint input exercises it together with the historical requirement.

Return a failed condition for refinement. When a source-validated check executes normally and fails on a candidate, the memory interface can return its unchanged program and actual failed observations for a further repair. The observer judges failures of the checked operation against the stated requirement. Passing conditions produce no failure feedback; failures in fixture preparation or observation remain unassessed. We compare this revision with reviewing the same requirement again.

3 HISTORY SUPPLIES A MISSING REPAIR REQUIREMENT

We use a SWE-bench Verified split with 280 source tasks, 70 development tasks, and a separate 150-task evaluation partition unused by these experiments. Table 1 shows three development comparisons in which memory changes whether the repair meets a requirement absent from the current issue. Django14404 receives the first-ranked source issue and patch. Django12708 and Django11138 come from a fixed six-task cohort selected by high Qwen similarity; its memory arm receives three unchanged Flat rules per task. Both arms resolve all six cohort tasks under official verification. The two displayed cases distinguish additional historical behaviors, rather than increasing that official score.

An omitted behavior of the same API. Django14404 reports that administrative redirects lose a deployment prefix. Both agents fix the prefix and pass official verification, but only the repair informed by source16612 also preserves the query string. The ordinary agent’s actual observations include the implementation that retains `QUERY_STRING`; seeing an available implementation does not by itself make query preservation part of its repair. A joint input combines the prefix with repeated and percent-encoded query values, `keep=a%2Fb&keep=c%20d`. Only the history-informed repair preserves both.

A neighboring operation. Django12708 reports ambiguous deletion of an ordinary index when a unique constraint uses the same columns. The first retrieved source, 15732, instead concerns deletion of a unique constraint sharing its column with a primary key. Both repairs fix ordinary-index selection; only the memory-informed repair also excludes primary keys when deleting unique constraints. The source-generated PostgreSQL check runs unchanged on both target repairs. A separate combined input creates an ordinary index, redundant uniqueness, and a primary key on the same column. Ordinary-index deletion already succeeds on this target-base input; both repairs retain it, but only the memory-informed repair also deletes redundant uniqueness while retaining the primary key and data. History supplies a requirement for the neighboring operation.

An independent argument of the same operation. Django11138 requires date conversions to respect the database’s source timezone. Source13363 requires `TruncDate` and `TruncTime` to honor an expression’s explicit destination timezone. Both agents repair the source timezone, but only the memory-informed repair also honors the destination. A source-generated check distinguishes the

Table 2: One fresh complete-agent attempt per input and known development task. All eight pass official verification. The two joint columns measure the original current/historical combinations. The MySQL SQL-generation and named-zone ORM cases are additional conditions specified after inspecting completed timezone repairs and then executed on every timezone submission.

Memory input	Constraint joint	Timezone joint	MySQL cases	Named-zone cases
None	Fail	Fail	Pass	Fail
Requirement	Fail	Pass	Pass	Pass
Implementation	Pass	Pass	Pass	Fail
Both	Pass	Pass	Fail	Pass

historical versions and runs unchanged on the target repairs. On an independent SQLite input combining a UTC+1 database with an explicit New York destination, only the memory-informed repair returns the correct date and time. The issue and history constrain different ends of the conversion.

The addition can guide further repair. Two fresh agents start from the same completed ordinary timezone patch. Issue-based additional review returns it unchanged; supplying a generated historical check and its failed observations leads the other agent to use `self.get_tzname()` in both truncation methods. Both revisions pass official verification, but only the latter passes the historical and joint inputs. The intervention supplies corrective information that further issue-based review does not recover in this comparison.

4 WHAT INFORMATION MAKES MEMORY USABLE?

A requirement can suffice without the source implementation. On the redirect task, replacing the first entry of the existing Flat packet with its original issue and patch yields a byte-identical repair; both forms preserve the joint behavior. A separate comparison starts two new agents from the completed ordinary patch. Both receive the source-generated sentence, “The redirect location from AdminSite.catch_all_view includes the original query string.” One also receives the generated check and its failed observations. Both submit byte-identical repairs that pass official and joint checks. The sentence-only agent finds `get_full_path()` in the current repository: memory communicates the desired behavior while the agent locates its implementation. Executable feedback adds no observed behavior in this pair.

Which information makes the requirement usable? We select the constraint and timezone benefit cases and run four fresh complete agents per task, fixing the single historical source and its delivery before editing. Two source-only constructions separate a behavioral requirement from an implementation reference; the combined input concatenates them unchanged. The reference also conveys behavior, so the comparison concerns these actual inputs rather than disjoint information categories. All eight repairs pass official verification (Table 2). On Django12708, only Implementation and Both also remove redundant uniqueness while preserving the primary key. The Requirement agent tests a field with `unique=True`, but places `unique_together` on a different, two-column set. This nearby passing test loses the same-column condition stated in memory. On Django11138, all three memory inputs preserve the explicit destination timezone, while the fresh no-memory repair omits it. The Requirement agent finds `self.get_tzname()` in the target implementation without being given that helper in memory.

The timezone repairs also differ on further conditions. Both still generates an unnecessary MySQL conversion between identical zones; Implementation instead misinterprets a named Paris source timezone, producing New York time 19:21 instead of 17:30 on a summer input. Requirement passes both additional diagnostics. These concrete omissions distinguish carrying a requirement, applying its triggering condition, and implementing the resulting behavior correctly. Two subsequent original-base agents receive the unchanged constraint Requirement, with or without a source-validated example; both pass official and joint checks. Thus the earlier failure does not establish that an implementation reference is necessary: the same requirement can yield different repairs depending on how its trigger is reconstructed.

	Official verification	Historical named tuples (2)	Named tuple + lazy User (1)	ADDITIONAL DIAGNOSTIC Ordinary tuple/list subclasses (2)
Memory supplied for revision				
Original issue + patch	Pass	Pass	Pass	Pass
Requirement sentence	Pass	Fail	Fail	Pass
Requirement + check + execution failures	Pass	Pass	Pass	Fail

Figure 1: Memory content changes which behaviors a repair preserves. One revision per input starts from the same Django12663 candidate and source13590. Parentheses give case counts; each cell reports all cases in its group. The ordinary-subclass inputs were chosen after inspecting the feedback repair. All three repairs pass official verification, while only original history passes every displayed behavior.

Execution feedback makes an omitted condition concrete. Two fresh agents refine the same original failed Requirement candidate, both receiving its requirement again. One also receives the unchanged source-generated check and its actual failures for a field with a primary key or `unique=True`. Requirement-only review excludes the primary key but still fails the ordinary unique-field condition. Feedback adds generated-name disambiguation and passes both conditions. Both revisions pass official verification and the withheld joint sequence, so the added behavior is recovery of the supplied condition, not an independent joint gain. The useful feedback identifies which stated condition the candidate still fails to implement.

5 APPLICABILITY IS PART OF REPAIR MEMORY

Recovering an example can overgeneralize its rule. On Django12663, both a three-source packet selected through code connections and a ten-source similarity packet contain source13590, yet neither repair recovers its named-tuple range behavior. Three subsequent fresh revisions hold that source and the initial candidate fixed. Figure 1 separates official success from recovery of the historical requirement and preservation of related inputs. Original history and execution feedback recover the named-tuple behavior, including its withheld combination with a lazy User; the requirement sentence alone does not. A subsequent review with the same budget and an empty reference returns the initial candidate unchanged and still fails these behaviors. The feedback repair, however, expands every non-builtin sequence into positional constructor arguments. This recovers named tuples but breaks ordinary tuple and list subclasses, which still work on the original-history repair, shared initial candidate and unmodified target. The missing distinction is the constructor’s calling convention: accepting a named tuple does not justify expanding every sequence subclass. Useful repair memory must retain the condition under which its example supports the implementation rule.

Relevant history need not add a missing requirement. In a fixed four-repository development cohort, sixteen original-base attempts compare no memory, Requirement, Implementation, and Both using the same top-ranked source per task. The first two inputs each resolve four of four tasks; the latter two each resolve three, omitting Xarray4687’s public `keep_attrs` argument. The unmodified Django15499 and Xarray4687 targets already pass their selected source examples for operation reduction and independent attribute mappings. These memories concern behavior already satisfied on the observed inputs.

Connect the historical condition to the current repair. A historical behavior missing from the target also need not improve its repair. We select two further development cases whose source-validated examples fail on their target versions, then compare no memory, original history, and a requirement with its source examples. All six repairs pass official verification; neither memory input improves the measured historical behavior on SymPy21379 or Astropy13236. Execution tracing separates the conditions: the SymPy history exercises `PolyElement.as_expr()`,

while the public issue enters `_parallel_poly_from_expr()`. In Astropy, both enter `_convert_data_to_col()`, but only the public structured-array input takes its `NdarrayMixin` conversion branch. These observations motivate selecting memory by the condition it contributes to the current repair; a shared function alone does not establish that connection.

We compare historical requirements with concrete edit advice on three further development tasks, holding the three retrieved sources per task fixed. Both forms resolve Django 16819 and Django 12858, and fail Xarray 7229: two of three official successes per arm. On Django 16819, both repairs incorrectly cancel `RemoveIndex` followed by `AddIndex` whenever the name is unchanged. A SQLite diagnostic, motivated by the first agent’s intermediate actions, exposes the lost effect: removing an index on `pink` and adding the same name on `weight` should change the indexed column. Both submitted repairs reduce this sequence to no operations, leaving `pink` indexed. The original version preserves the replacement; both repairs also preserve the add-then-remove and unchanged-definition comparison cases.

The matched comparison on Xarray 7229 illustrates excessive preservation. Both agents repair the common coordinate-attribute reproduction and pass all 280 official pass-to-pass tests, but fail the official target test. The edit-advice repair retains the condition array’s coordinate attributes when both value arguments to `where` are scalars; the requirements repair already does so when only the first is scalar. Neither memory form establishes the boundary between required preservation and required change in these repairs.

That boundary can already be wrong in the historical memory. We generate source-only contracts with applicability, intended-change and preservation fields. For Xarray 4094, the contract claims a stack/unstack roundtrip preserves sample coordinates when variables have only sample dimensions. The literal official source patch restores the reported three-element dataset, but a manually chosen one-element input loses the sample dimension and coordinate. The original source raises a merge error on both inputs. The singleton satisfies the generated condition, so this package is rejected before target use. Explicit contract fields alone do not establish the scope supported by the repaired source.

6 RELATED WORK

ExpGraph learns graph retrieval from task scores with and without a retrieved experience packet (Feng et al., 2026). RSCB-MC selects memory injection, retrieval variants or abstention from contextual compatibility and uncertainty signals (Iskan, 2026). Our interventions measure source-specific behaviors that can differ while the official target score stays unchanged. ExpeL distills reusable insights from experience (Zhao et al., 2024). ExpeRepair uses historical test demonstrations and reflective insights to generate reproduction and boundary tests (Mu et al., 2026). MIRA separates SQL corrections into reusable requirements and uses database checks to guide their activation and adaptation (Liu et al., 2026). TRACE compiles user corrections into atomic rules and runtime checks for later coding-agent tasks (Zhou et al., 2026b). Villuri et al. (2026) combine task-derived dependency graphs and structured execution feedback for coupled code synthesis; their ablation separates feedback from the graph contribution. These methods establish historical examples, requirements and executable checks as tools for agent learning. Our content and refinement interventions isolate how an agent can receive the relevant requirement yet lose one of its conditions when implementing it. Working-set interventions study how supplying required facts supports code generation and how written conventions can override working code (Mohammadi et al., 2026). SWE-RPG supplies requirement and planning references for retrospective diagnosis of complete coding-agent trajectories (Zhou et al., 2026a). Our development comparison examines when a retrieved historical requirement helps preserve current and historical behaviors together, including behavior absent from the official target tests.

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing.

REFERENCES

- Tao Feng, Chongrui Ye, Tianyang Luo, Jingjun Xu, Xueqiang Xu, Haozhen Zhang, Zhigang Hua, Yan Xie, Shuang Yang, Ge Liu, and Jiaxuan You. ExpGraph: Model-agnostic experience learning with graph-structured memory for LLM agents, 2026. URL <https://arxiv.org/abs/2605.30712v1>.
- Mehmet Iscan. Learning when to remember: Risk-sensitive contextual bandits for abstention-aware memory retrieval in llm-based coding agents, 2026. URL <https://arxiv.org/abs/2604.27283v1>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Yining Liu, Chenyu Yang, Boyan Li, Rui Mao, and Yuyu Luo. MIRA: Evidence-verified repair memory for text-to-sql correction, 2026. URL <https://arxiv.org/abs/2608.06950v1>.
- Bardia Mohammadi, Lars Klein, Aman Chadha, Akhil Arora, and Laurent Bindschaedler. The working set of a coding agent: Coherence debt in repository-scale tasks, 2026. URL <https://arxiv.org/abs/2608.16630v1>. Version 1.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair, 2026. URL <https://arxiv.org/abs/2506.10484v2>. Version 2; accepted at FSE 2026.
- Gnaneswar Villuri, Hashmath Shaik, and Alex Doboli. Dynamic adaptation of the llm context for generating routines with coupled semantics, 2026. URL <https://arxiv.org/abs/2609.04570v1>.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models, 2025. URL <https://arxiv.org/abs/2506.05176v3>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144v3>.
- Xin Zhou, Chun Yong Chong, Kisub Kim, Yun Peng, Rui Shu, Zihan Wu, Xu Han, Guowen Yuan, Zeyang Zhuang, Jounghoon Kim, Jeongjin Ju, Seongmin Ju, Taein Yoon, and David Lo. A unified issue resolution benchmark for requirement clarification, planning, and code generation for coding agents, 2026a. URL <https://arxiv.org/abs/2608.09072v1>.
- Yujun Zhou, Kehan Guo, Haomin Zhuang, Xiangqi Wang, Yue Huang, Zhenwen Liang, Pin-Yu Chen, Tian Gao, Nuno Moniz, Nitesh V. Chawla, and Xiangliang Zhang. Getting better at working with you: Compiling user corrections into runtime enforcement for coding agents, 2026b. URL <https://arxiv.org/abs/2606.13174v1>.